C# Console RPG 開発設計書

文書名：C# Console RPG 要件定義書・基本設計書
バージョン：1.0
開発言語：C#
開発環境：Visual Studio
アプリケーション形式：コンソールアプリケーション
開発方式：オブジェクト指向設計
作成日：2026年11月9日

第1章　プロジェクト概要

1.1 開発目的

C#を使用して、コマンド選択式のターン制RPGを開発する。

本開発では、単純にゲームを完成させることだけでなく、オブジェクト指向プログラミング、クラス設計、責務分離、インターフェース、継承、ポリモーフィズムなどの技術を実践的に習得することを目的とする。

また、将来的な機能追加を考慮し、拡張性・保守性を重視した設計を行う。

1.2 ゲーム概要

プレイヤーが複数の敵キャラクターとターン制バトルを行い、敵を撃破することで経験値やアイテムを獲得するRPGとする。

プレイヤーは戦闘を通じてレベルアップし、能力値を成長させながらゲームを進行する。

1.3 開発方針

以下の方針に基づいて開発する。

1. クラスごとの責務を明確にする。
2. 共通処理は適切に抽象化する。
3. クラス間の依存関係を必要最小限にする。
4. 新しい敵・スキル・アイテムを追加しやすい構造にする。
5. ゲーム進行処理と画面表示処理を分離する。
6. 各クラスの単体テストが可能な構造にする。

第2章　システム要件定義

2.1 機能要件

ID	機能名	要件内容
FR-01	ゲーム開始	ゲームを開始し、プレイヤーを生成する
FR-02	プレイヤー管理	名前・HP・攻撃力・防御力・レベル等を管理する
FR-03	敵管理	複数種類の敵キャラクターを生成・管理する
FR-04	ステータス管理	キャラクターの能力値と現在の状態を管理する
FR-05	戦闘開始	プレイヤーと敵の戦闘を開始する
FR-06	コマンド選択	プレイヤーが戦闘中の行動を選択する
FR-07	通常攻撃	対象にダメージを与える
FR-08	防御	次の敵行動に対する被ダメージを軽減する
FR-09	回復	HPを回復する

FR-10	スキル	MP等を消費して特殊効果を発動する
FR-11	アイテム	所持アイテムを使用する
FR-12	ダメージ計算	攻撃力・防御力等を基にダメージを算出する
FR-13	敵AI	敵の行動を決定する
FR-14	ターン管理	プレイヤーと敵の行動順を管理する
FR-15	勝敗判定	戦闘の勝利・敗北を判定する
FR-16	経験値管理	敵撃破時に経験値を付与する
FR-17	レベルアップ	必要経験値に達した場合、能力値を成長させる
FR-18	装備管理	武器・防具等の装備状態を管理する
FR-19	アイテム管理	アイテムの所持数・使用状態を管理する
FR-20	ゲーム終了	ゲーム終了処理を行う

2.2 非機能要件

ID	要件名	内容
NFR-01	拡張性	新しい敵・スキル・アイテムを追加しやすくする
NFR-02	保守性	クラスごとの責務を明確にする
NFR-03	再利用性	共通処理を複数のクラスから利用できるようにする
NFR-04	整合性	HP・MP・所持数等の不正な状態を防止する
NFR-05	テスト容易性	各クラスを独立してテストできるようにする
NFR-06	可読性	クラス名・メソッド名から処理内容を理解できるようにする
NFR-07	保守容易性	仕様変更による修正範囲を限定する

2.3 システム制約

- C#を使用する。
- Visual Studioで開発する。
- コンソールアプリケーションとして実装する。
- 外部ゲームエンジンは使用しない。
- 初期段階ではネットワーク通信を実装しない。
- 初期段階ではデータベースを使用しない。
- セーブ・ロード機能は後続開発で実装する。

第3章　ゲーム仕様

3.1 戦闘方式

完全ターン制を採用する。

1ターンの基本的な処理順序は以下とする。

1. プレイヤーの行動選択
2. プレイヤーの行動実行
3. 敵の生存状態確認
4. 敵の行動決定
5. 敵の行動実行
6. プレイヤーの生存状態確認
7. 戦闘継続判定
8. 次のターンへ移行

プレイヤーが敵を全滅させた場合は勝利とする。

プレイヤーのHPが0になった場合は敗北とする。

3.2 プレイヤーの行動

以下の6種類を実装対象とする。

行動	内容	制約
通常攻撃	敵1体に攻撃する	対象が生存していること
防御	次の敵攻撃の被ダメージを軽減する	防御状態を管理する
回復	自身のHPを回復する	最大HPを超えない
スキル	MP等を消費して特殊効果を発動する	必要なリソースを保持していること

アイテム	所持アイテムを使用する	所持数が1以上であること
逃走	戦闘から離脱する	成功・失敗判定を行う

3.3 キャラクターのステータス

項目	説明
Name	キャラクター名
Level	レベル
MaxHp	最大HP
CurrentHp	現在HP
MaxMp	最大MP
CurrentMp	現在MP
AttackPower	攻撃力
DefensePower	防御力
Speed	素早さ
Experience	経験値

※すべてのキャラクターが全項目を使用するかは、詳細設計で確定する。

3.4 ダメージ計算

基本ダメージ計算式を以下とする。

基本ダメージ ＝ 攻撃側の攻撃力 − 防御側の防御力

最終ダメージは最低1とする。

最終ダメージ ＝ Math.Max(1, 基本ダメージ)

今後、属性・クリティカル・スキル補正等を追加できる構造とする。

3.5 成長システム

敵撃破時に経験値を獲得する。

経験値が規定値に達した場合、レベルアップする。

レベルアップ時には以下の能力値を成長させる。

- 最大HP
- 最大MP
- 攻撃力
- 防御力

成長量および必要経験値の計算式は詳細設計で決定する。

3.6 敵の行動

敵はEnemyAIによって行動を決定する。

初期段階では以下を実装する。

- 通常攻撃
- 防御
- 特定条件でのスキル使用

複数の敵が存在する場合、それぞれの敵が独立して行動する。

第4章　クラス構成・責務定義

4.1 クラス一覧

No.	クラス名	役割
1	GameManager	ゲーム全体の進行管理
2	BattleManager	戦闘全体の進行管理
3	Character	キャラクター共通情報の管理
4	Player	プレイヤー固有情報の管理
5	Enemy	敵固有情報の管理
6	DamageCalculator	ダメージ計算
7	EnemyAI	敵の行動決定
8	BattleAction	戦闘行動の共通仕様
9	AttackAction	通常攻撃処理
10	DefenseAction	防御処理
11	HealAction	回復処理
12	SkillAction	スキル処理
13	ItemAction	アイテム使用処理

14	Inventory	アイテム所持管理
15	Equipment	装備情報管理
16	ExperienceManager	経験値・成長管理
17	ConsoleUI	入力・画面表示管理

※上記は論理的な責務の一覧であり、すべてを独立クラスにするかは詳細設計で判断する。

4.2 GameManager

【目的】

ゲーム全体の進行を管理する。

【主な責務】

- ゲーム開始
- プレイヤー生成
- ゲーム進行状態の管理
- 戦闘開始の指示
- ゲーム終了処理

【責務範囲外】

- ダメージ計算
- 個々のキャラクターのHP管理
- 敵AIの行動決定

4.3 BattleManager

【目的】

戦闘処理を統括する。

【主な責務】

- 戦闘開始・終了
- ターン管理
- 行動順の管理
- プレイヤー行動の実行指示
- 敵行動の実行指示

- 勝敗判定

【責務範囲外】

- 画面への直接的な文字出力
- 個々のダメージ計算式の保持
- アイテムデータの定義

4.4 Character

【目的】

プレイヤーと敵に共通するキャラクター情報を定義する。

【主な責務】

- 基本ステータスの管理
- HP・MPの管理
- ダメージ処理
- 回復処理
- 生存状態の判定

【設計方針】

抽象クラスとして定義することを検討する。

HP等の状態変更は適切なメソッドを経由させ、外部からの不正な変更を防止する。

4.5 Player

【目的】

プレイヤー固有の情報を管理する。

【主な責務】

- レベル管理
- 経験値管理
- インベントリの保持
- 装備情報の保持
- プレイヤー固有の成長処理

Characterを継承することを基本方針とする。

4.6 Enemy

【目的】

敵キャラクター固有の情報を管理する。

【主な責務】

- 敵固有の能力値管理
- 撃破時の経験値報酬管理
- 撃破時のアイテム報酬管理
- 敵固有の属性・特性管理

Characterを継承することを基本方針とする。

4.7 DamageCalculator

【目的】

ダメージ計算処理を担当する。

【主な責務】

- 基本ダメージ計算
- 防御力による軽減
- 属性補正
- クリティカル補正

計算処理と実際のHP変更処理を分離する。

4.8 EnemyAI

【目的】

敵の行動内容を決定する。

【主な責務】

- 敵の行動選択
- プレイヤーの状態確認
- スキル使用条件の判定
- 行動対象の決定

4.9 BattleAction

【目的】

戦闘中の各種行動に共通する仕様を定義する。

【主な責務】

- 行動の実行
- 行動対象の指定
- 行動結果の返却

通常攻撃・防御・回復・スキル等を統一的に扱える構造を検討する。

4.10 ConsoleUI

【目的】

コンソール画面の表示とユーザー入力を担当する。

【主な責務】

- ステータス表示
- コマンド表示
- ユーザー入力の取得
- 戦闘ログ表示
- 入力値の検証

ゲームの内部ロジックと画面表示処理を分離する。

第5章　クラス間の関係

5.1 継承関係

Characterを基底クラスとし、PlayerとEnemyを派生クラスとして定義する。

Character
├── Player
└── Enemy

5.2 主要な依存関係

GameManager
└── BattleManager

BattleManager
├── Player
├── Enemy（複数）
├── EnemyAI
├── BattleAction
└── ConsoleUI

BattleAction
├── AttackAction
├── DefenseAction
├── HealAction
├── SkillAction
└── ItemAction

DamageCalculator
└── ダメージ計算を必要とする処理から利用

Player
├── Inventory
├── Equipment
└── ExperienceManager（利用方式は詳細設計で確定）

5.3 設計上の注意

各クラスが他クラスの内部状態を直接変更することを避ける。

必要な処理は公開メソッドやインターフェースを通じて実行する。

また、ConsoleUIがゲームの戦闘ロジックを直接実行しないようにする。

第6章　データ管理方針

6.1 キャラクターデータ

キャラクターの基本能力値と現在状態を区別して管理する。

例：

- 最大HP：100
- 現在HP：75

最大HPと現在HPを別々に管理し、回復・ダメージ処理で整合性を維持する。

6.2 アイテムデータ

アイテムは種類・効果・所持数を管理する。

同一種類のアイテムを複数所持できるようにする。

6.3 装備データ

武器・防具等の装備情報を管理する。

装備による能力値補正は、基礎能力値と区別して扱う方針とする。

6.4 データ保存

初期実装ではメモリ上で管理する。

後続開発でJSON形式による保存・読み込み機能を追加する。

第7章　テスト方針

7.1 単体テスト

各クラスの機能を独立して確認する。

例:

- Characterのダメージ処理
- Characterの回復処理
- DamageCalculatorの計算結果
- EnemyAIの行動決定
- ExperienceManagerのレベルアップ判定

7.2 異常系テスト

以下のケースを確認する。

- HPが0未満にならないこと
- HPが最大HPを超えないこと
- 負数のダメージを受け付けないこと
- 所持数が0のアイテムを使用できないこと
- MP不足時にスキルを使用できないこと
- 死亡したキャラクターが行動できないこと
- 無効なコマンド入力でゲームが異常終了しないこと

7.3 結合テスト

複数クラスを連携させ、戦闘全体の動作を確認する。

- プレイヤー攻撃から敵HP減少まで
- 敵攻撃からプレイヤーHP減少まで
- 敵撃破から経験値獲得まで
- レベルアップから能力値変更まで
- 全敵撃破から戦闘終了まで

第8章　開発工程

フェーズ	作業内容	成果物
1	要件定義	要件定義書
2	基本設計	クラス一覧・クラス図
3	詳細設計	メソッド仕様書・プロパティ仕様書
4	キャラクター実装	Character・Player・Enemy
5	戦闘システム実装	BattleManager・DamageCalculator
6	行動システム実装	BattleAction関連クラス
7	成長システム実装	経験値・レベルアップ
8	アイテム・装備実装	Inventory・Equipment
9	テスト	単体・結合テスト
10	統合	RPG完成版

第9章　今後の設計課題

以下の項目は、詳細設計に進む前に確定する。

1. プレイヤーと敵のステータス項目の確定
2. HP・MPの変更ルール
3. 防御状態の有効期間
4. スキルの種類と消費MP
5. アイテムの効果と使用条件

6. 複数の敵がいる場合のターゲット選択
7. 敵の行動順
8. 逃走成功率の計算方法
9. 経験値とレベルアップの計算式
10. 装備による能力値補正方法
11. 状態異常・属性システムの実装範囲
12. 戦闘中の死亡・復活の扱い

第10章　設計完了条件

以下を満たした段階で実装工程へ移行する。

- 機能要件が確定している。
- 非機能要件が確定している。
- 各クラスの責務が定義されている。
- クラス間の依存関係が整理されている。
- プロパティとメソッドの仕様が定義されている。
- 正常系・異常系の動作が定義されている。
- テスト条件が明確になっている。

本設計書は初版であり、詳細設計時に必要に応じて更新する。